Enclave Collector System - Complete Function Tree

This document provides a comprehensive function tree showing the complete call hierarchy and error paths for the enclave-collector system. It documents exactly where execution starts and which functions are called throughout the system's operation.

📋 Table of Contents

1. Main Entry Points
2. System Initialization Flow
3. Vote Processing Flow
4. API Integration Flow
5. Error Handling Paths
6. Configuration Management
7. Enclave Operations
8. Network Operations
9. Cleanup and Shutdown

Main Entry Points

🚀 Primary Execution Entry Point

```
main() [host_main.c:208]
├── signal(SIGINT, signal_handler)
├── signal(SIGTERM, signal_handler)
├── init_default_config(&g_host_context.config)
├── parse_arguments(argc, argv, &g_host_context.config)
│   ├── → [SUCCESS] Continue execution
│   ├── → [1] Exit (help/version displayed)
│   └── → [ERROR_INVALID_PARAMETER] Return error
├── load_config_file(g_host_context.config.config_file, &g_host_context.config)
│   ├── → [SUCCESS] Configuration loaded
│   └── → [ERROR] Return error
├── logging_init(&g_host_context.config)
│   ├── → [SUCCESS] Logging initialized
│   └── → [ERROR] Return error
├── host_initialize(&g_host_context)
│   ├── → [SUCCESS] Host interface ready
│   └── → [ERROR] Goto cleanup
├── run_collector_service(&g_host_context)
│   ├── → [SUCCESS] Service completed
│   └── → [ERROR] Service failed
└── cleanup:
    ├── host_cleanup(&g_host_context)
    ├── logging_cleanup()
    └── return result
```

System Initialization Flow

🔧 Host Interface Initialization

```
host_initialize(host_context_t* context) [host_interface.c:34]
├── memset(context, 0, sizeof(host_context_t))
├── context->session_id = (uint64_t)time(NULL)
├── config_system_init("./config/enclave_config.json")
│   ├── config_load_from_file(config_file_path, &g_system_config)
│   │   ├── fopen(config_file_path, "r")
│   │   ├── fread() - Read JSON configuration
│   │   ├── extract_config_string() - Parse JSON fields
│   │   ├── extract_config_int() - Parse numeric fields
│   │   └── → [ENCLAVE_SUCCESS/WARNING] Configuration loaded
│   └── config_load_environment(&g_system_config)
│       ├── getenv("API_BASE_URL")
│       ├── getenv("API_AUTH_TOKEN")
│       └── → Override file settings with env vars
├── config_get_current()
│   └── → Return pointer to g_system_config
├── api_client_init(&system_config->api_config)
│   ├── memcpy(&g_api_config, config, sizeof(api_config_t))
│   ├── [Windows] WSAStartup(MAKEWORD(2, 2), &wsaData)
│   ├── [Linux] curl_global_init(CURL_GLOBAL_DEFAULT)
│   └── → [ENCLAVE_SUCCESS] API client ready
├── file_operations_init(&context->config)
│   └── → [SUCCESS] File operations ready
├── [SIMULATION_MODE] sim_initialize_collector(&context->collector_state)
│   ├── memset(&g_sim_state, 0, sizeof(collector_state_t))
│   ├── sim_generate_keys(&g_sim_keys)
│   │   ├── Generate simulation private key
│   │   ├── Generate simulation public key
│   │   └── → [SUCCESS] Keys generated
│   ├── g_sim_initialized = 1
│   └── → [SUCCESS] Simulation ready
├── [PRODUCTION] host_create_enclave(context)
│   ├── oe_create_collector_enclave()
│   │   ├── → [OE_OK] Enclave created
│   │   └── → [Error] Return ERROR_INITIALIZATION_FAILED
│   └── host_initialize_enclave(context)
│       ├── ecall_initialize_collector()
│       └── → [SUCCESS/ERROR] Enclave initialized
└── context->is_initialized = 1
```

🌐 Service Startup

```
run_collector_service(host_context_t* context) [host_main.c:147]
├── network_initialize(&net_config, &server)
│   ├── Set server configuration
```

```
│   └── → [ENCLAVE_SUCCESS] Network initialized
├── network_start_server(&server)
│   ├── Create listening socket
│   ├── Bind to port
│   ├── Start listening
│   └── → [ENCLAVE_SUCCESS] Server started
├── Main service loop:
│   │   while (g_running) {
│   │       Sleep(1000) / sleep(1)
│   │       // Periodic maintenance tasks
│   │   }
├── network_stop_server(&server)
│   └── → Close connections and stop server
└── network_cleanup(&server)
    └── → Clean up network resources
```

Vote Processing Flow

Vote Submission Processing

```
host_process_request(host_context_t* context, host_request_t* request,
host_response_t* response) [host_interface.c:220]
├── host_validate_request(request)
│   ├── Check for null pointers
│   ├── Check message size limits
│   ├── Validate timestamp tolerance
│   └── → [SUCCESS/ERROR] Request validated
├── memset(response, 0, sizeof(host_response_t))
├── Switch (request->message_type):
│   ├── MSG_TYPE_VOTE_SUBMISSION:
│   │   ├── Validate payload size >= sizeof(vote_t)
│   │   ├── host_handle_vote_submission(context, vote, &vote_id)
│   │   │   ├── [SIMULATION] sim_process_vote(vote, vote_id)
│   │   │   │   ├── Basic vote validation:
│   │   │   │   │   ├── Check vote_id not empty
│   │   │   │   │   ├── Check candidate_id > 0
│   │   │   │   │   ├── Check timestamp within tolerance
│   │   │   │   │   └── → [ERROR_*] Validation failed
│   │   │   │   ├── sim_verify_signature()
│   │   │   │   │   ├── Create expected checksum
│   │   │   │   │   ├── Extract signature checksum
│   │   │   │   │   └── → [SUCCESS] if checksums match
│   │   │   │   ├── Assign vote_id = g_next_vote_id++
│   │   │   │   ├── g_sim_state.total_votes++
│   │   │   │   ├── g_sim_state.valid_votes++
│   │   │   │   └── → [SUCCESS] Vote processed
│   │   │   └── [PRODUCTION] ecall_process_vote()
│   │   │       ├── Make ECALL to enclave
│   │   │       └── → [SUCCESS/ERROR] Enclave result
│   │   ├── Allocate response data
```

```
│   │   ├── Copy vote_id to response
│   │   └── → [SUCCESS] Response prepared
│   ├── MSG_TYPE_AGGREGATION_REQUEST:
│   │   ├── host_handle_aggregation_request(context, &summary)
│   │   │   ├── [SIMULATION] sim_aggregate_votes(summary)
│   │   │   │   ├── Calculate candidate distributions
│   │   │   │   ├── Generate aggregation proof
│   │   │   │   └── → [SUCCESS] Aggregation complete
│   │   │   └── [PRODUCTION] ecall_aggregate_votes()
│   │   └── → Return aggregation summary
│   └── MSG_TYPE_STATUS_REQUEST:
│       ├── host_handle_status_request(context, &state)
│       └── → Return collector state
└── → [SUCCESS] Request processed
```

API Integration Flow

External API Operations

```
API Client Operations [api_client.c]
├── api_client_init(api_config_t* config)
│   ├── memcpy(&g_api_config, config)
│   ├── [Windows] WSAStartup()
│   ├── [Linux] curl_global_init()
│   └── → [ENCLAVE_SUCCESS] Client ready
├── Election Parameter Fetching:
│   │   api_get_election_parameters(election_params_t* params)
│   │   ├── http_get("/api/election/parameters", &response)
│   │   │   ├── curl_easy_init() / Windows socket operations
│   │   │   ├── curl_easy_setopt() / Set request headers
│   │   │   ├── curl_easy_perform() / Send request
│   │   │   └── write_callback() - Receive response data
│   │   ├── Parse JSON response (mock data):
│   │   │   ├── params->num_candidates = 3
│   │   │   ├── params->max_votes = 1000
│   │   │   ├── Set election timeframe
│   │   │   └── Generate dummy public key
│   │   └── → [ENCLAVE_SUCCESS] Parameters fetched
├── Auxiliary Values Processing:
│   │   api_fetch_auxiliary_values(election_id, &values, &count)
│   │   ├── http_get("/api/auxiliary/fetch")
│   │   ├── Allocate auxiliary_value_t array
│   │   ├── Fill with mock data
│   │   └── → [ENCLAVE_SUCCESS] Values ready
├── Vote Result Submission:
│   │   api_submit_vote_result(election_id, receipt)
│   │   ├── Create JSON payload with vote data
│   │   ├── http_post("/api/collector/vote-result", json_data)
│   │   └── → [ENCLAVE_SUCCESS] Result submitted
├── Aggregation Storage:
```

```
│   │   api_store_aggregation_result(election_id, result)
│   │   ├── Format result as JSON
│   │   ├── http_post("/api/results/store", json_data)
│   │   └── → [ENCLAVE_SUCCESS] Result stored
└── Key Management:
    ├── api_store_enclave_key(key_id, key) [Disabled]
    └── api_fetch_enclave_key(key_id, key) [Disabled]
```

🔧 HTTP Operations

```
HTTP Request Processing [api_client.c]
├── http_get(endpoint, response)
│   ├── Construct full URL from base_url + endpoint
│   ├── [Linux] curl_easy_init()
│   │   ├── curl_easy_setopt(CURLOPT_URL, url)
│   │   ├── curl_easy_setopt(CURLOPT_WRITEFUNCTION, write_callback)
│   │   ├── curl_easy_setopt(CURLOPT_TIMEOUT, timeout)
│   │   ├── curl_easy_perform(curl)
│   │   └── curl_easy_cleanup(curl)
│   ├── [Windows] Manual socket operations
│   │   ├── socket(AF_INET, SOCK_STREAM, 0)
│   │   ├── connect() to server
│   │   ├── send() HTTP request
│   │   └── recv() HTTP response
│   └── → [ENCLAVE_SUCCESS/ERROR] HTTP complete
├── http_post(endpoint, json_data, response)
│   ├── Similar to http_get with POST method
│   ├── Add Content-Type: application/json header
│   ├── Send JSON payload in request body
│   └── → [ENCLAVE_SUCCESS/ERROR] POST complete
└── write_callback(contents, size, nmemb, response)
    ├── Calculate total_size = size * nmemb
    ├── realloc(response->data, response->size + total_size + 1)
    ├── memcpy(response->data + response->size, contents, total_size)
    ├── response->size += total_size
    └── → Return total_size
```

Error Handling Paths

⚠ Error Propagation Flow

```
Error Handling Throughout System:
├── Configuration Errors:
│   │   config_system_init() failure
│   │   ├── Log warning about config file
│   │   ├── Fall back to environment variables
│   │   ├── Fall back to default values
│   │   └── Continue execution (non-fatal)
```

```
├── Initialization Errors:
│   │   host_initialize() failure
│   │   ├── api_client_init() failure
│   │   │   ├── [Windows] WSAStartup failure
│   │   │   ├── [Linux] curl_global_init failure
│   │   │   └── → Return ENCLAVE_ERROR_NETWORK_INITIALIZATION_FAILED
│   │   ├── file_operations_init() failure
│   │   │   └── → Return ERROR_INITIALIZATION_FAILED
│   │   ├── [SIMULATION] sim_initialize_collector() failure
│   │   │   ├── sim_generate_keys() failure
│   │   │   └── → Return ERROR_*
│   │   └── [PRODUCTION] host_create_enclave() failure
│   │       ├── oe_create_collector_enclave() failure
│   │       │   └── → Return ERROR_INITIALIZATION_FAILED
│   │       └── host_initialize_enclave() failure
│   │           ├── ecall_initialize_collector() failure
│   │           └── → Return ERROR_INITIALIZATION_FAILED
├── Runtime Errors:
│   │   Vote Processing Errors:
│   │   ├── ERROR_INVALID_VOTE_ID
│   │   ├── ERROR_INVALID_CANDIDATE_ID
│   │   ├── ERROR_VOTE_EXPIRED
│   │   ├── ERROR_VERIFICATION_FAILED
│   │   └── → Log error and return to client
│   │   Network Errors:
│   │   ├── ERROR_NETWORK_FAILED
│   │   ├── ERROR_CONNECTION_FAILED
│   │   ├── ERROR_NETWORK_TIMEOUT
│   │   └── → Retry or fail gracefully
│   │   API Errors:
│   │   ├── ENCLAVE_ERROR_API_NOT_INITIALIZED
│   │   ├── ENCLAVE_ERROR_NETWORK_REQUEST_FAILED
│   │   ├── ENCLAVE_ERROR_JSON_PARSE_ERROR
│   │   └── → Log and continue with fallbacks
├── Memory Errors:
│   │   malloc() / realloc() failures
│   │   ├── ERROR_OUT_OF_MEMORY
│   │   ├── ENCLAVE_ERROR_MEMORY_ALLOCATION
│   │   └── → Clean up partial allocations and fail
└── Cleanup Errors:
    │   Shutdown sequence errors
    │   ├── host_destroy_enclave() failure
    │   │   ├── oe_terminate_enclave() failure
    │   │   └── → Log warning, continue cleanup
    │   ├── network_cleanup() failure
    │   │   └── → Log warning, continue cleanup
    │   └── api_client_cleanup() failure
    │       ├── [Windows] WSACleanup()
    │       ├── [Linux] curl_global_cleanup()
    │       └── → Complete cleanup regardless
```

Configuration Management

Configuration Loading Flow

```
Configuration System [config_manager.c]
├── config_system_init(config_file_path)
│   ├── config_load_from_file(config_file_path, &g_system_config)
│   │   ├── fopen(config_file_path, "r")
│   │   │   ├── → [SUCCESS] File opened
│   │   │   └── → [FAIL] Log warning, use defaults
│   │   ├── fread() - Read entire file content
│   │   ├── extract_config_string(content, "api_base_url")
│   │   │   ├── Find key in JSON content
│   │   │   ├── Extract quoted string value
│   │   │   └── → Return allocated string or NULL
│   │   ├── extract_config_int(content, "api_timeout_ms", default)
│   │   │   ├── Find key in JSON content
│   │   │   ├── Parse integer value
│   │   │   └── → Return parsed value or default
│   │   ├── Parse all configuration fields:
│   │   │   ├── api_base_url
│   │   │   ├── api_auth_token
│   │   │   ├── api_timeout_ms
│   │   │   ├── api_max_retries
│   │   │   ├── log_level
│   │   │   └── data_directory
│   │   └── → [ENCLAVE_SUCCESS] Configuration parsed
│   ├── config_load_environment(&g_system_config)
│   │   ├── getenv("API_BASE_URL")
│   │   ├── getenv("API_AUTH_TOKEN")
│   │   ├── getenv("API_TIMEOUT_MS")
│   │   ├── getenv("LOG_LEVEL")
│   │   ├── getenv("DATA_DIRECTORY")
│   │   └── → Override file config with env vars
│   └── g_config_loaded = 1
├── config_get_current()
│   ├── Check g_config_loaded
│   └── → Return &g_system_config
├── config_get_api_base_url()
│   └── → Return g_system_config.api_config.base_url
└── config_validate()
    ├── Validate API base URL format
    ├── Validate timeout values > 0
    ├── Validate retry counts >= 0
    └── → [ENCLAVE_SUCCESS] Configuration valid
```

Enclave Operations

Enclave Lifecycle Management

```
Enclave Operations [host_interface.c]
├── [PRODUCTION MODE] Enclave Management:
│   │   host_create_enclave(context)
│   │   ├── oe_create_collector_enclave(
│   │   │       enclave_path,
│   │   │       OE_ENCLAVE_TYPE_SGX,
│   │   │       OE_ENCLAVE_FLAG_DEBUG,
│   │   │       &context->enclave_handle)
│   │   ├── → [OE_OK] Enclave created successfully
│   │   └── → [Error] Log error and return failure
│   │   host_initialize_enclave(context)
│   │   ├── ecall_initialize_collector(
│   │   │       enclave_handle,
│   │   │       &ecall_result,
│   │   │       &context->collector_state)
│   │   ├── → [OE_OK] ECALL succeeded
│   │   └── → [Error] Log error and return failure
│   │   host_destroy_enclave(context)
│   │   ├── oe_terminate_enclave(context->enclave_handle)
│   │   ├── context->enclave_handle = NULL
│   │   └── → [SUCCESS] Enclave terminated
├── [SIMULATION MODE] Simulation Operations:
│   │   sim_initialize_collector(state)
│   │   ├── memset(&g_sim_state, 0, sizeof(collector_state_t))
│   │   ├── sim_generate_keys(&g_sim_keys)
│   │   │   ├── Generate mock private key
│   │   │   ├── Generate mock public key
│   │   │   └── → [SUCCESS] Keys generated
│   │   ├── g_sim_initialized = 1
│   │   └── → [SUCCESS] Simulation ready
│   │   sim_process_vote(vote, vote_id)
│   │   ├── Validate vote fields
│   │   ├── sim_verify_signature()
│   │   ├── Assign unique vote_id
│   │   ├── Update vote counters
│   │   └── → [SUCCESS] Vote processed
│   │   sim_aggregate_votes(summary)
│   │   ├── Calculate vote distributions
│   │   ├── Generate mock proof
│   │   └── → [SUCCESS] Aggregation complete
└── Vote Processing ECALLs (Production):
    ├── ecall_process_vote(enclave, vote, vote_id)
    ├── ecall_aggregate_votes(enclave, summary)
    ├── ecall_get_collector_state(enclave, state)
    └── → [OE_OK/Error] Enclave operation result
```

Network Operations

🌐 Network Interface Operations

```
Network Operations [network_interface.c]
├── network_initialize(net_config, server)
│   ├── Validate configuration parameters
│   ├── Initialize server structure
│   ├── Set socket options
│   └── → [ENCLAVE_SUCCESS] Network ready
├── network_start_server(server)
│   ├── Create listening socket
│   │   ├── socket(AF_INET, SOCK_STREAM, 0)
│   │   ├── setsockopt(SO_REUSEADDR)
│   │   └── → Socket created
│   ├── Bind to specified port
│   │   ├── bind(socket, address, sizeof(address))
│   │   └── → [SUCCESS] Bound to port
│   ├── Start listening for connections
│   │   ├── listen(socket, max_connections)
│   │   └── → [SUCCESS] Listening started
│   ├── Accept incoming connections
│   │   ├── accept(socket, client_addr, &addr_len)
│   │   ├── Create client handler thread
│   │   └── → Continue accepting
│   └── → [ENCLAVE_SUCCESS] Server running
├── network_handle_client(client_socket)
│   ├── Receive request data
│   │   ├── recv(client_socket, buffer, size, 0)
│   │   └── → Parse incoming data
│   ├── Process request through host_process_request()
│   ├── Send response data
│   │   ├── send(client_socket, response, size, 0)
│   │   └── → Response sent
│   └── close(client_socket)
├── network_stop_server(server)
│   ├── Set shutdown flag
│   ├── Close listening socket
│   ├── Wait for client threads to finish
│   └── → [ENCLAVE_SUCCESS] Server stopped
└── network_cleanup(server)
    ├── Free allocated resources
    ├── Close remaining sockets
    └── → [SUCCESS] Cleanup complete
```

Cleanup and Shutdown

Graceful Shutdown Flow

```
Cleanup and Shutdown [host_main.c:cleanup]
├── Signal Handler (SIGINT/SIGTERM):
│   │   signal_handler(signal)
│   │   ├── printf("Received signal %d, shutting down gracefully...")
```

```
│   │   ├── g_running = 0
│   │   └── → Exit main service loop
├── Service Shutdown:
│   │   run_collector_service() cleanup
│   │   ├── network_stop_server(&server)
│   │   │   ├── Close listening socket
│   │   │   ├── Terminate client connections
│   │   │   └── → [SUCCESS] Network stopped
│   │   └── network_cleanup(&server)
│   │       └── → Free network resources
├── Host Interface Cleanup:
│   │   host_cleanup(&g_host_context)
│   │   ├── [PRODUCTION] host_destroy_enclave(context)
│   │   │   ├── oe_terminate_enclave(enclave_handle)
│   │   │   ├── context->enclave_handle = NULL
│   │   │   └── → [SUCCESS] Enclave terminated
│   │   ├── [SIMULATION] Cleanup simulation state
│   │   │   ├── g_sim_initialized = 0
│   │   │   ├── memset(&g_sim_state, 0)
│   │   │   └── memset(&g_sim_keys, 0)
│   │   ├── file_operations_cleanup()
│   │   │   └── → Close file handles and cleanup
│   │   ├── api_client_cleanup()
│   │   │   ├── [Windows] WSACleanup()
│   │   │   ├── [Linux] curl_global_cleanup()
│   │   │   └── → Network library cleanup
│   │   └── context->is_initialized = 0
├── Logging Cleanup:
│   │   logging_cleanup()
│   │   ├── Flush log buffers
│   │   ├── Close log file handles
│   │   └── → [SUCCESS] Logging cleaned up
└── Final Cleanup:
    ├── Free any remaining allocated memory
    ├── Reset global state variables
    └── → Return exit code
```

📊 Summary Statistics

Function Count by Module:

- **host_main.c**: 8 functions (main entry point, argument parsing, service loop)
- **host_interface.c**: 45+ functions (core host operations, simulation, enclave management)
- **api_client.c**: 20+ functions (HTTP operations, API integration)
- **config_manager.c**: 15+ functions (configuration loading and management)
- **network_interface.c**: 10+ functions (network server operations)
- **file_operations.c**: 8+ functions (file I/O operations)
- **logging.c**: 6+ functions (logging system)

Error Paths Documented:

- **91 distinct error codes** across multiple categories
- **Initialization errors**: Configuration, network, enclave, API
- **Runtime errors**: Vote processing, validation, memory allocation
- **Network errors**: Connection failures, timeouts, protocol errors
- **Cleanup errors**: Resource cleanup failures, shutdown issues

External Dependencies:

- **Open Enclave SDK** (production mode)
- **libcurl** (Linux HTTP operations)
- **Winsock2** (Windows network operations)
- **Standard C library** (all platforms)

🏗 Architecture Notes

Design Patterns Used:

1. **Dual-mode operation**: Simulation vs. Production enclave execution
2. **External API integration**: All election parameters and results stored externally
3. **Error propagation**: Consistent error handling with cleanup paths
4. **Resource management**: Proper allocation/deallocation with cleanup handlers
5. **Configuration hierarchy**: File → Environment → Defaults

Security Considerations:

1. **Enclave isolation**: Sensitive operations performed in hardware enclave
2. **External key storage**: No static cryptographic material in enclave
3. **Auxiliary value processing**: External API provides additional vote data
4. **Signature verification**: Both simulation and production signature checking
5. **Secure communication**: HTTPS for external API calls (configurable)

This function tree provides a complete view of the enclave-collector system's execution flow, error handling, and architectural patterns. Each branch shows the exact function call hierarchy and error propagation paths throughout the system.